As an alternative to specifying an end point `DateTime`, you can simply provide the number of required iterations. The following code produces identical output to the example above:

```
$period = new DatePeriod(
    $bellRiots,
    new DateInterval('PT12H'),
    4
);
```

Note that the start point is automatically included, and the end point is not. This behavior can be controlled with a pair of constants – `DatePeriod::EXCLUDE_START_DATE` and `DatePeriod::INCLUDE_END_DATE` – supplied as a fourth argument.

# Functional Interface

The `date` module also provides a functional API. Certain parts of the functional API predate the OOP (PHP 4 vs. the object API introduction in PHP 5.2), and most of the `date` functions are wrappers around the OOP stuff. Quite a few of them will return some form of object instance. The functions that are OOP wrappers are given below:

***Table 9-7.*** *`date` extension functions (OOP wrappers)*

| Function | OOP Alias |
|---|---|
| `date()` | `new DateTime($timestampString)->format()` |
| `date_add` | `DateTime::add()` |
| `date_create` | `new DateTime()` |
| `date_create_from_format()` | `DateTime::createFromFormat()` |
| `date_create_immutable` | `new DateTimeImmutable()` |
| `date_create_immutable_from_format()` | `DateTimeImmutable::createFromFormat()` |
| `date_date_set()` | `DateTime::setDate()` |
| `date_diff()` | `DateTime::diff()` |

(*continued*)